
Web Development and Design

Lecture 04: Client-Server Web Applications

What is a Web Client?

- Clients are PCs or workstations on which users run applications. Clients rely on servers for resources, such as files, devices, and even processing power.
- **Web Clients** are applications that communicate with a web server using hypertext transfer protocol. *Web Browsers* are examples of web clients: Mozilla Firefox, Internet explorer, Google Chrome, Apple Safari, Opera, Netscape Navigator...
- **Client can be**
 - Thin client; or
 - Fat client

What is a Web Client?

- **A fat client** (also called heavy, rich or thick client) is a computer (client) in client server architecture or networks that typically provides rich functionality independent of the central server
 - Eg. SMTP,FTP,DNS Server
- **A thin client** (sometimes also called a lean, zero or slim client) is a computer or a program or an application that depends heavily on another computer (its server) to fulfill its computational roles.
 - Eg. Thunderbird, MS Outlook, Remote Desktop

What is a Server?

- Servers are powerful computers or processes dedicated to managing resources e.g. disk drives (file servers), printers (print servers), or network traffic (network servers).
- **Web Servers** a special type of server (remote machines) that stores and delivers web pages to users i.e a piece of software designed to serve web pages/web sites/ web services: Apache, WAMP...
- **Server can be**
 - **Static server**
 - **Dynamic servers**

What is a Server?

- **Static server** (delivering pre-built content):
 - **Content Delivery:** Serves pre-generated HTML files directly to the client (web browser).
 - **Content Updates:** Requires manual modification of the underlying files on the server to update content.

What is a Server?

■ Static server

• Advantages:

- **Faster loading times:** Since the content is pre-built, there's no processing required on the server, making them ideal for simple websites with static content like brochures or portfolios.
- **Easier to manage:** Requires less technical knowledge to set up and maintain.
- **More secure:** Less vulnerable to attacks as they don't process user input or access databases.

• Disadvantages:

- **Limited interactivity:** Cannot offer dynamic features like user logins, shopping carts, or real-time updates.
- **Less scalable:** Updating content across multiple pages can be tedious and time-consuming.

Client/Server Model

- **Dynamic servers** (generating content on-the-fly using server-side scripting):
 - **Content Delivery:** Generates HTML on-the-fly based on user requests and server-side scripting languages like PHP, Python, or Java.
 - **Content Updates:** Updates can be made dynamically through a content management system (CMS) or by modifying server-side scripts.

Client/Server Model

- **Dynamic servers**

- **Advantages:**

- **High interactivity:** Can offer features like user logins, personalized content, shopping carts, and real-time updates.
- **Highly scalable:** Easier to manage content across multiple pages due to dynamic generation.
- **More flexible:** Can adapt to different user requests and situations.

Client/Server Model

- **Dynamic servers**

- **Disadvantages:**

- **Slower loading times:** Processing requests and generating content on-the-fly can take more time compared to static servers.
- **More complex to manage:** Requires knowledge of server-side scripting languages and database management.
- **More vulnerable to attacks:** Have a larger attack surface due to the use of scripting languages and databases.

Network Models

- A network model is a design specification for **how the nodes on a network interact and communicate**. The network model determines the degree to which **communications and processing are centralized or distributed**.
- There are three primary network models:
 - Centralized or hierarchical
 - Client/server
 - Peer-to-peer

Centralized Model

- **Centralized model:**

- This model emphasizes the **centralized nature of the architecture, where a single server acts as the main point of control and communication.**
- It can be implemented using various architectural styles, including the client-server model.
- The centralized model might also consider aspects like **data storage and processing logic** being located on the central server.

Centralized Model

- **Use Cases for the Centralized Model:**
 - **Simple Websites:** Ideal for static websites or websites with limited functionality and user interaction.
 - **Small Business Applications:** Suitable for internal applications used within a small organization where scalability needs are moderate.
 - **Landing Pages:** Often used for promotional campaigns or lead generation, where simplicity and quick deployment are priorities.

Client/Server Model

- This model emphasizes the **communication flow and interaction between clients (web browsers) and servers.**
- **Structure:** It defines a specific communication pattern where **clients send requests to servers, and servers process them and send responses back.**
- **Flexibility:** This model can be implemented in various configurations, including **centralized architectures** with a single server, or **decentralized architectures** with multiple servers distributed across different locations.

Client/Server Model

- **Client (Web Browser):** Represents the user's device (computer, phone, etc.) running a web browser.
- **Central Server:** Represents the single server responsible for handling all client requests and delivering responses.
- **Internet:** Represents the network connection that allows communication between the client and the server.

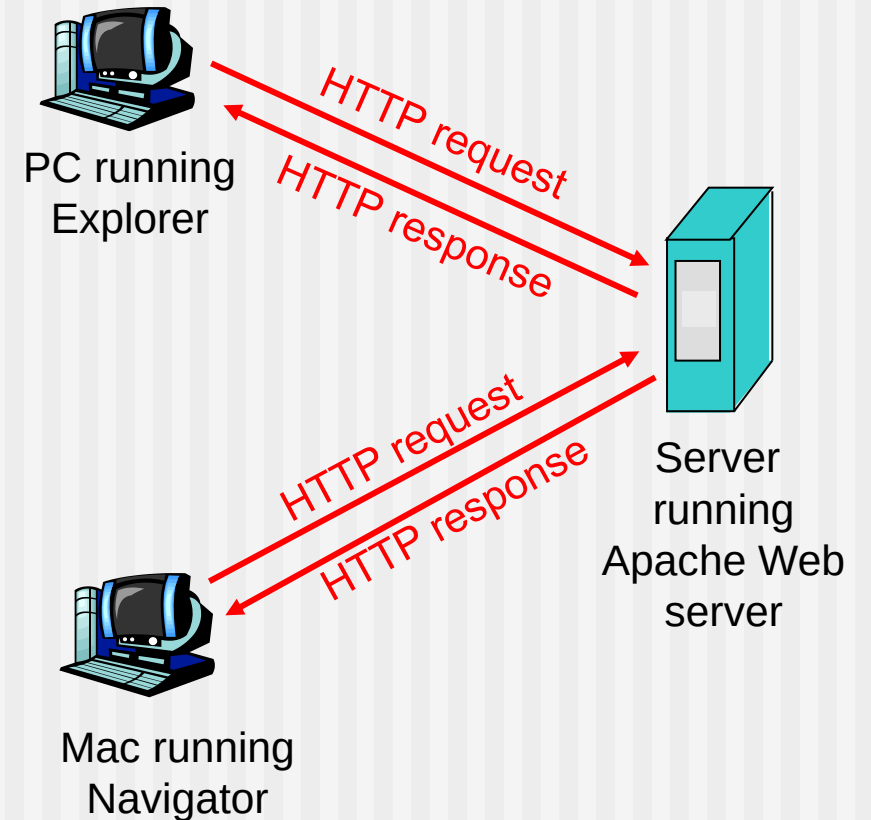
Client/Server Model

- Clients and servers need to understand each other, so they use a common language called a "**protocol**".
- **Web Protocols:** These are protocols that are specifically designed for communication on the World Wide Web.
 - **HTTP (Hypertext Transfer Protocol):** This is the protocol used for communication **between web browsers and web servers**.
 - **HTTPS (Hypertext Transfer Protocol Secure):** A secure version of HTTP that encrypts communication between browser and server, safeguarding data privacy and integrity.

HTTP overview

HTTP: hypertext transfer protocol

- client/server model
 - *client*: browser that requests, receives, “displays” Web objects
 - *server*: Web server sends objects in response to requests
- **Example:**
 - You (client) type a website address into your browser (client).
 - Your browser sends an HTTP request to the web server.
 - The web server sends back the website's HTML, CSS, and images (response).



Client/Server Model

- **Internet Protocols:** govern communication across the entire internet, not just the web.
 - **TCP/IP (Transmission Control Protocol/Internet Protocol):** A suite of protocols that govern **how data is broken down into packets, transmitted over the internet, and reassembled at the destination.** Understanding TCP/IP is crucial for troubleshooting network connectivity issues.
 - **SMTP (Simple Mail Transfer Protocol):** Used for sending email.
 - **DNS (Domain Name System):** Used to translate domain names into IP addresses.
- While all web protocols are internet protocols, not all internet protocols are web protocols.

Client/Server Model in a Step-by-Step

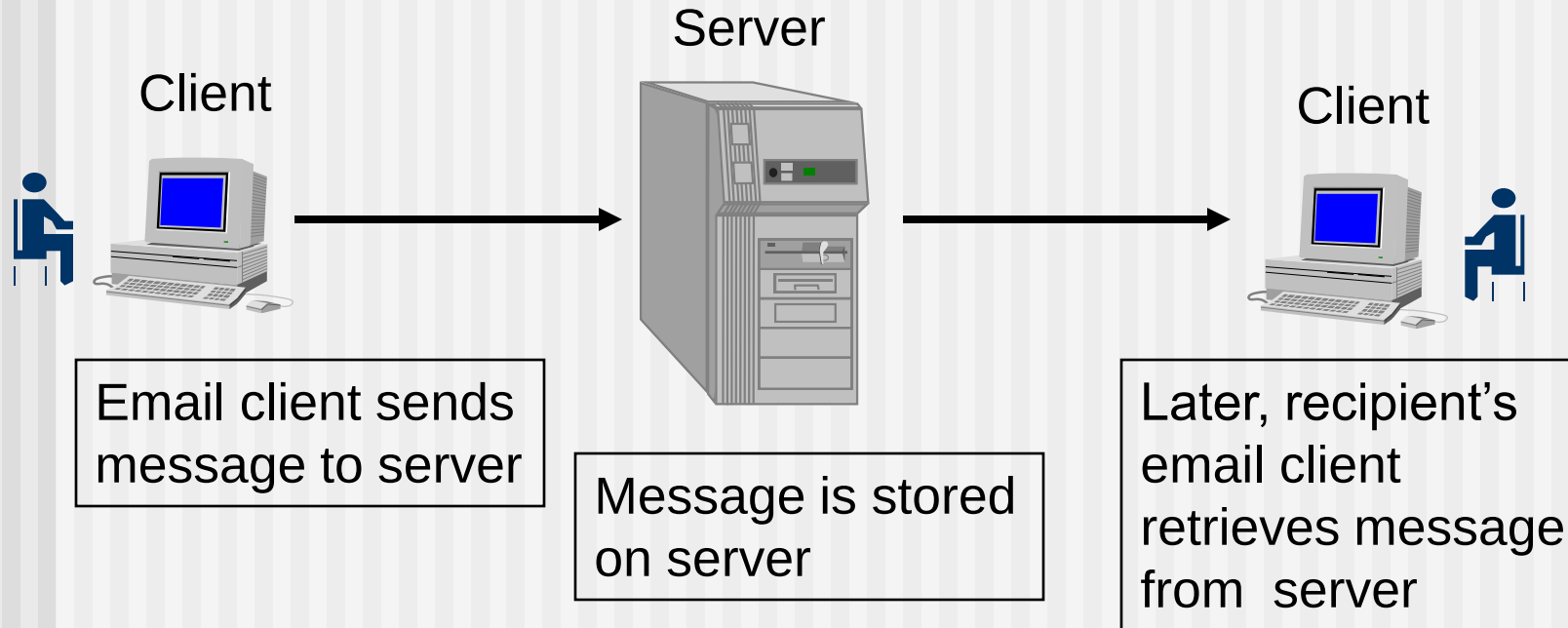
- 1. Client Request:** The client sends a request to the central server, specifying the desired web page or resource.
- 2. Server Processing:** The central server receives the request, processes it (retrieving data, generating content, etc.), and prepares a response.
- 3. Server Response:** The central server sends the response back to the client, containing the requested web page, data, or error message.
- 4. Web Page Display:** The client's web browser receives and interprets the response, displaying the web page or content to the user.

Client/Server Model in a Step-by-Step

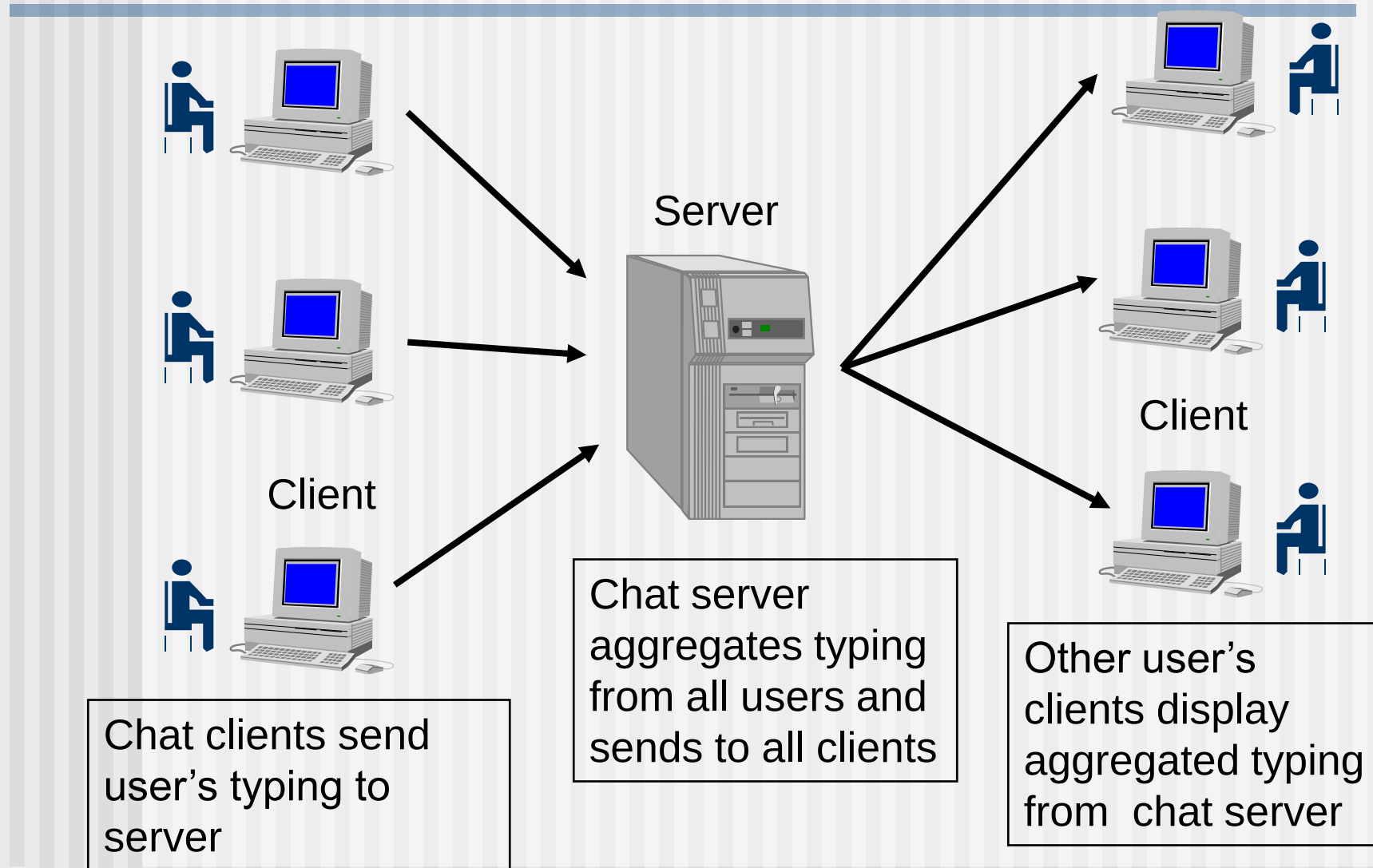
- **Additional elements:**

- **Database:** The centralized model often utilizes a database on the server to store website data (text, images, user information, etc.).
- **Application Logic:** Business logic (processing tasks) can be implemented on the central server, determining how data is manipulated and responses are generated.
 - **Example:** Amazon's server has logic that calculates shipping costs, applies discounts, and processes payments.

Example: Email application



Example: Chat application



Client/Server Model Advantages

1. **Organization:** It facilitates clear organization of data and resources, making them easier to manage and maintain.
2. **Scalability:** Centralized servers can be scaled up or down to accommodate changing traffic demands.
3. **Security:** By controlling access and permissions on the central server, you can enhance security measures.

Disadvantages of a Client/Server Network

This structure offers several **disadvantages** for web development:

- **Expense:** Requires initial investment in dedicated server.
- **Maintenance:** Large networks will require a staff to ensure efficient operation.
- **Dependence:** When server goes down, operations will cease across the network

Client/Server Model vs Centralized Model

- Every **centralized model** is a **client-server model**, but **not every client-server model is a centralized model**. Client-server model can be decentralized model with multiple servers distributed across different locations.
- The **centralized model** highlights the **single point of control** and centralized location, while the **client-server model** focuses on the **communication flow** between clients and servers.

Peer-to-Peer Model

- Another type of network architecture is known as **a peer-to peer architecture** because each node has equivalent responsibilities.
- All clients on the network are equal in terms of providing and using resources and users are authenticated by each individual workstations.
- Easy and inexpensive model in terms of configuration and implementation. However, they are **only practical in very small organizations due to lack of central data storage and administration.**

Peer-to-Peer Model

- **No Central Server:** In a P2P network, there isn't one central server that controls everything. Instead, each computer (or 'node') acts as both a client (requesting resources) and a server (providing resources).
- **Everyone Connects:** Nodes connect directly to each other, forming a network of peers. They discover each other through various methods, often involving some initial coordination.

Peer-to-Peer Model

- **Resource Sharing:** Nodes share resources directly with each other. This could be files (like music or movies), processing power (for distributed computing tasks), or even internet bandwidth.
- **Search and Discovery:** P2P networks often have search mechanisms. When you want a specific file, your computer searches the network to find other nodes that have it.
- **Example:** Think of a file-sharing application. Instead of downloading a movie from a central server, you download tiny pieces of the file from many different users on the P2P network who already have them.

Peer-to-Peer Model

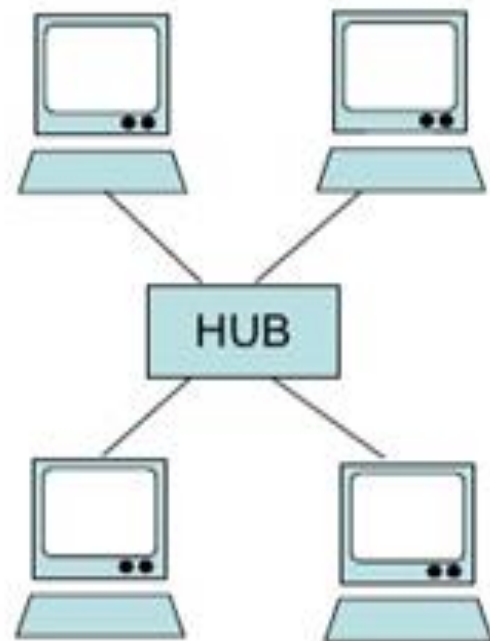
Advantages of a peer-to-peer Network

- **Less initial expense** - No need for a dedicated server.
- **Setup** - An operating system already in place may only need to be reconfigured for peer-to-peer operations.

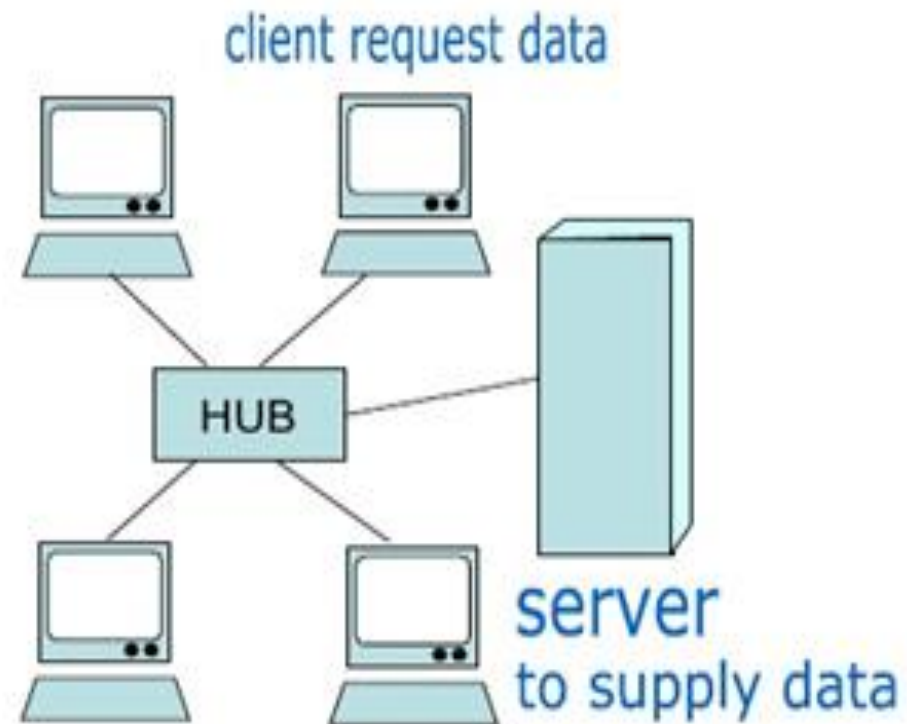
Disadvantages of a Peer-to-Peer Network

- ***Decentralized*** - No central repository for files and applications.
- ***Security*** - Does not provide the security available on a client/server network.

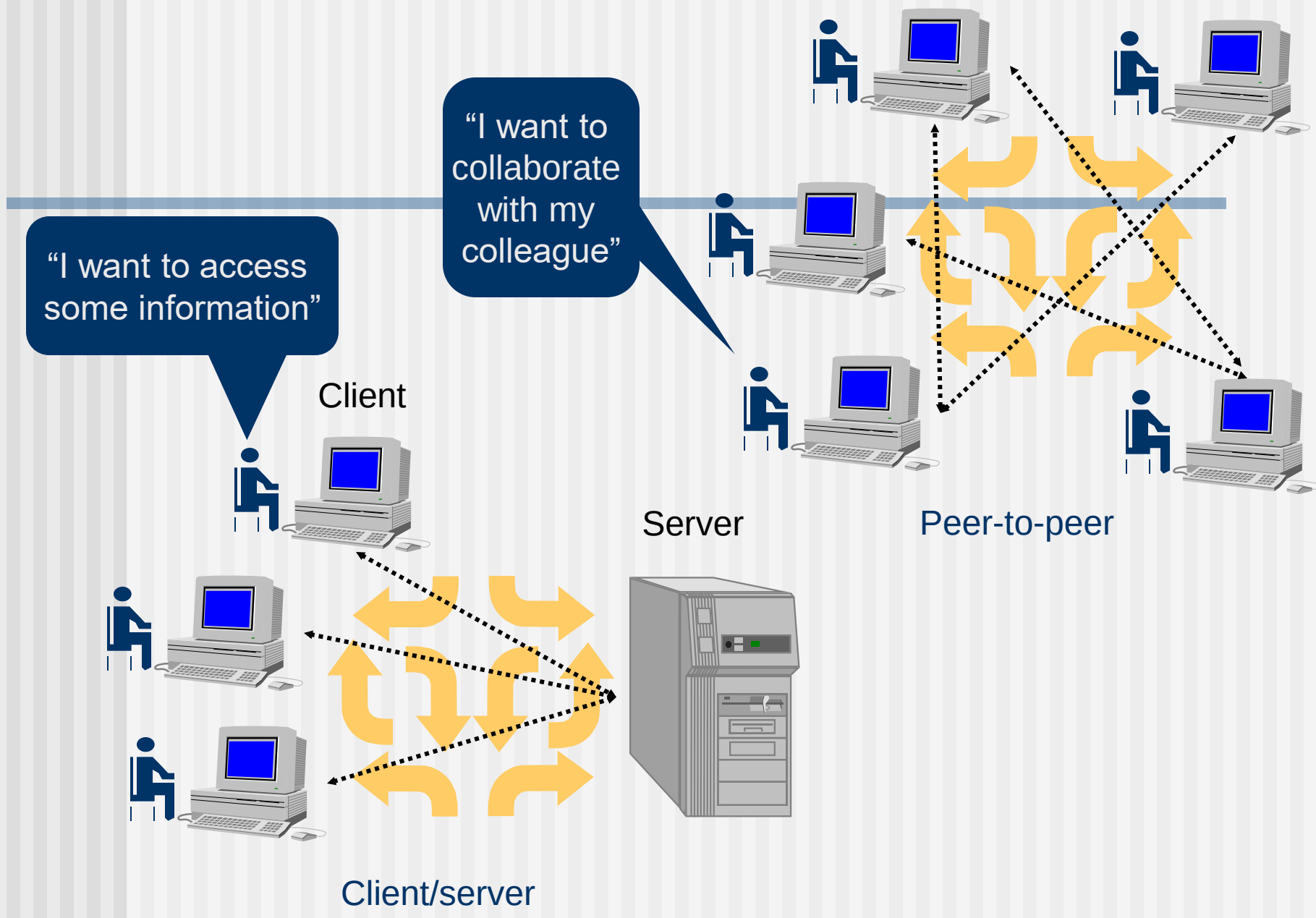
ARCHITECTURE



peer to peer



client - server



What is Client Server Architecture?

- determines how web applications are structured and organized.
- The application is modeled as a set of services that are provided by servers and a set of clients that use these services.
- **Types of Client Server Architecture:**
 - **Two-Tier Architecture**
 - **Three-Tier Architecture**
 - **Distributed Architecture**
 - **N-Tier Architecture**

2-Tier Architecture

- The Two-tier architecture is divided into two parts:
 - Client Application (Client Tier)
 - Database (Data Tier)
- For e.g.: On client application side the code is written for saving the data in the SQL server database. Client sends the request to server, and it process the request & send back with data. The main problem of two-tier architecture is the server cannot respond multiple request same time, as a result it cause a data integrity issue.

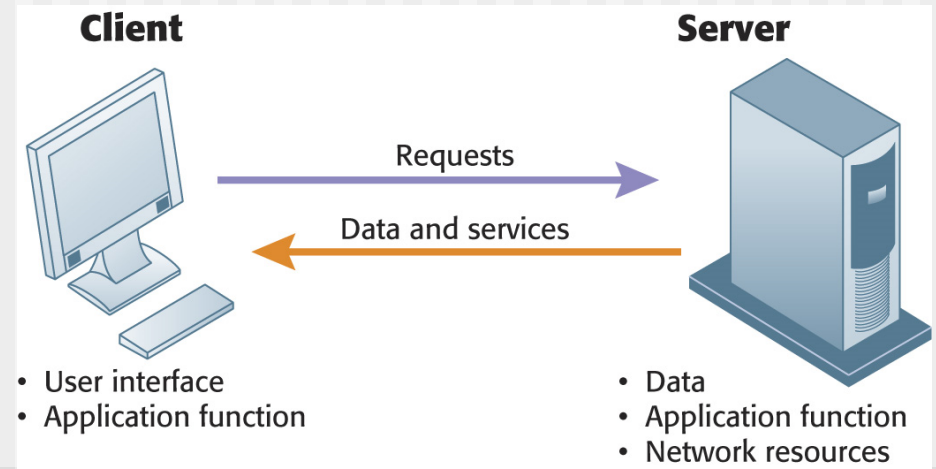
2-Tier Architecture

- **Simpler structure:** Consists of two layers: a presentation layer (client-side) and a data layer (server-side).
- **Combined logic:** Business logic (processing tasks) can reside in either the client-side or server-side layer, or both.
- **Suited for:** Simpler applications or applications with limited functionality.

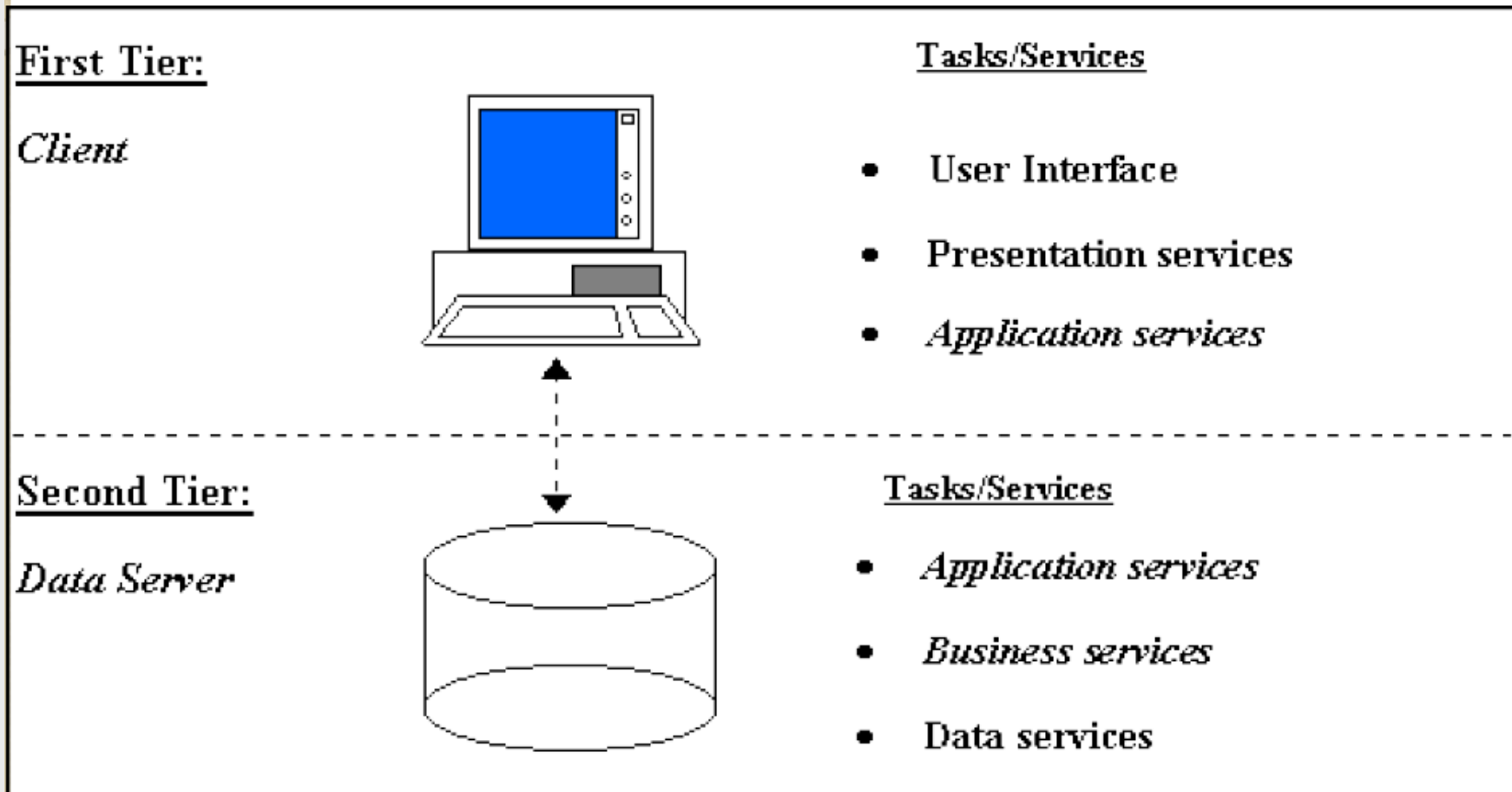
2-Tier Architecture

■ 2-Tier Architecture Example:

- **Simple Calculator Web App:** Imagine a basic calculator web app that allows users to perform simple arithmetic operations. The user interface (HTML, CSS, JavaScript) would reside on the client-side, while the server-side script (Python, PHP, etc.) would handle user input, perform calculations, and send the results back to the client.



2-Tier Architecture



2-Tier Architecture

- **Advantages:**

- Easy to maintain and modification is bit easy
- Communication is faster

- **Disadvantages:**

- In two tier architecture application performance will be degrade upon increasing the users.

3-Tier Architecture

- Each of the application layers may execute on a separate processor.
- Allows for better performance than a thin-client model and is simpler to manage than a fat-client model.
- A more scalable architecture – as demands increase, extra servers can be added.

3-Tier Architecture

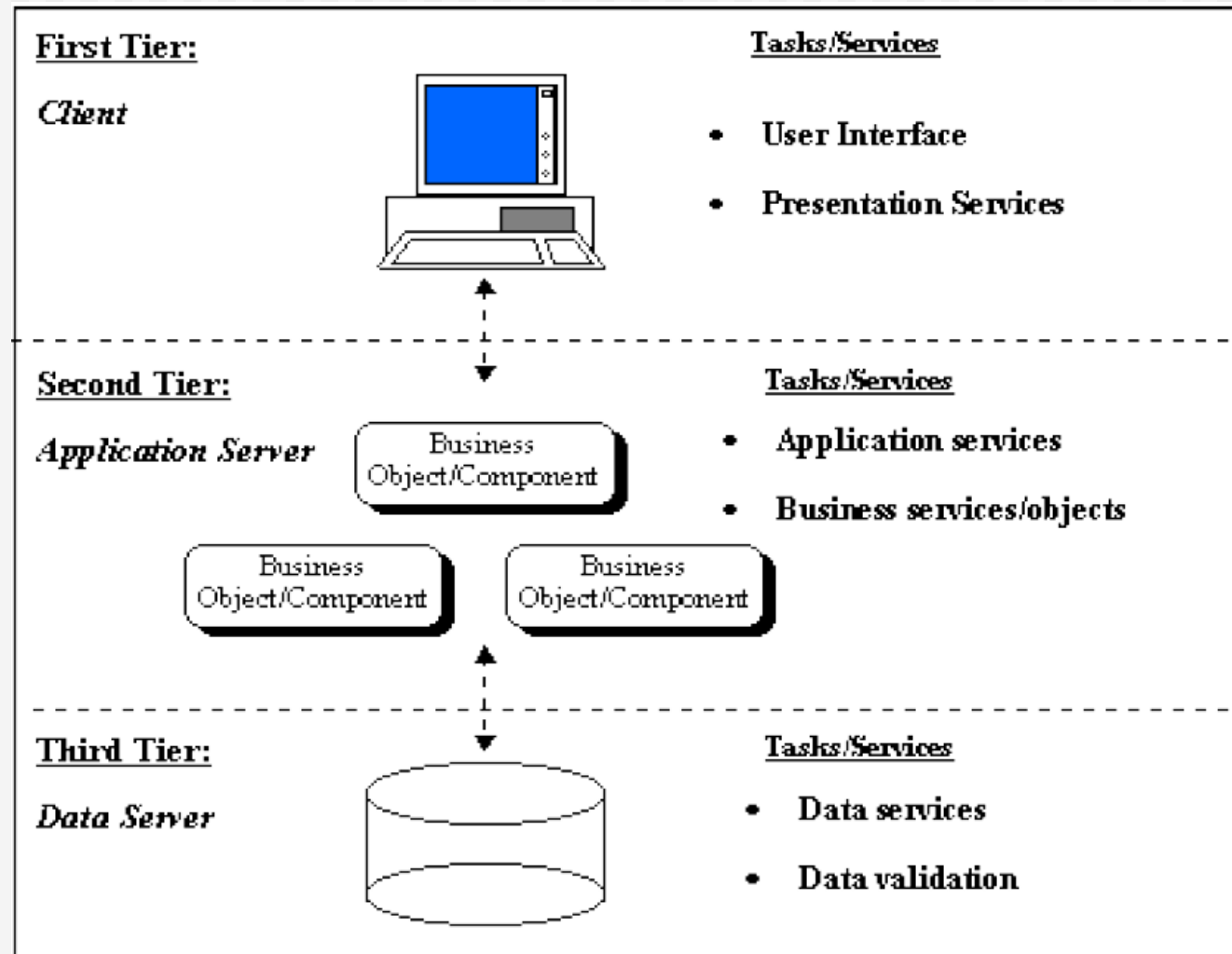
- **Modular structure:** Introduces a separate middle layer (business layer) between the presentation and data layers.
- **Dedicated logic:** Business logic is isolated in the middle layer, promoting code reusability and maintainability.
- **Improved scalability and security:** Enables easier scaling and enhanced security measures due to separation of concerns.

3-Tier Architecture

■ 3-Tier Architecture Example:

- **E-commerce Website:** An e-commerce website involves complex functionalities like product catalogs, user accounts, shopping carts, and payment processing:
 - **Presentation layer:** The user interface (web pages) built with HTML, CSS, and JavaScript would handle user interactions and display product information.
 - **Business layer:** This layer, implemented in a server-side language like Java or Python, would handle business logic like product data retrieval, order processing, and payment authorization. It would interact with the database layer.
 - **Data layer:** The database (MySQL, PostgreSQL, etc.) would store product data, user information, and order details. The business layer would interact with the database to perform CRUD (create, read, update, delete) operations.

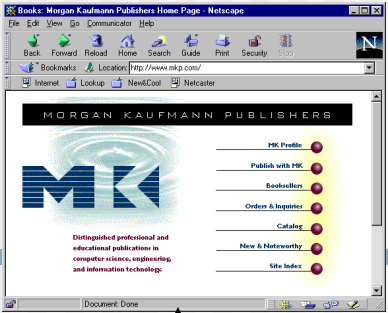
3-Tier Architecture



Host architecture

Client

Web browser



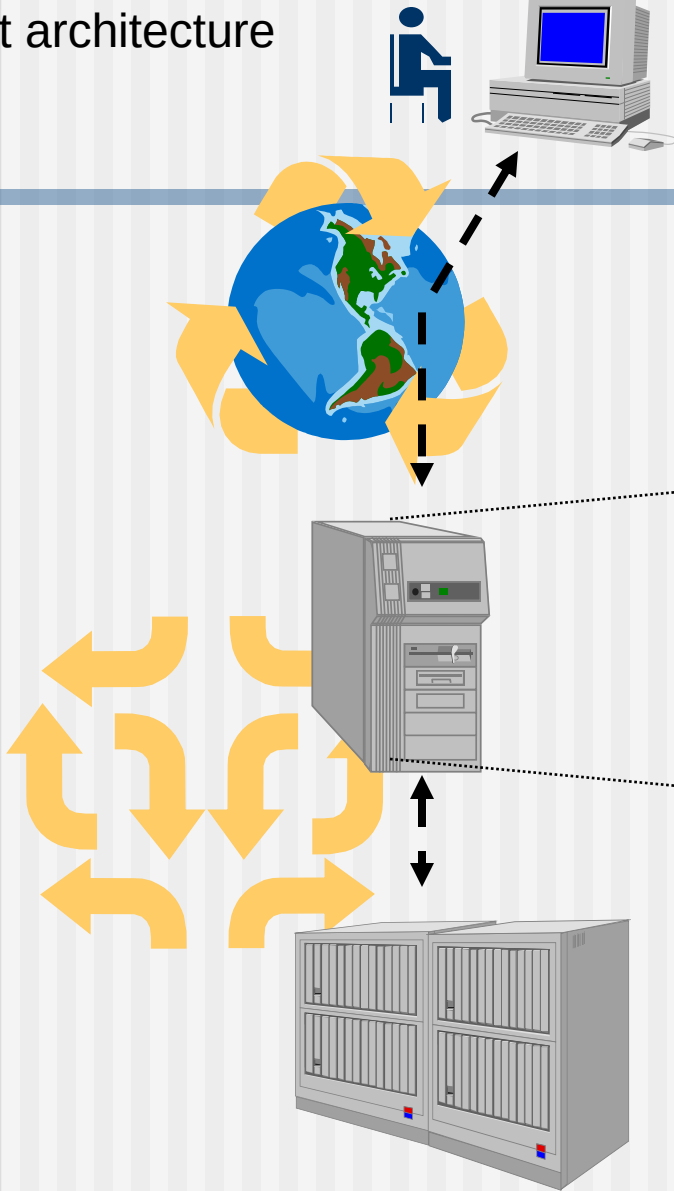
Web server

Application logic

Databases and DBMS

Common gateway interchange

Application partition



Types of Client Server Architecture

- The choice between two-tier and three-tier architectures depends on the complexity and requirements of your web application.
 - **Two-tier:** Suitable for smaller-scale projects where simplicity and ease of development are priorities.
 - **Three-tier:** Preferred for complex applications requiring better organization, maintainability, security, and scalability.

N-Tier Architecture

- This is a **generalization** of the three-tier model, allowing for **additional layers** to be introduced as needed.
- Each layer can have dedicated functionalities and responsibilities, further enhancing the **flexibility and scalability** of the architecture.
- However, it also increases **complexity** and requires careful design and implementation.

N-Tier Architecture

- In complex e-commerce platforms, an additional tier might be introduced, such as:
 - **Service layer:** Acts as an abstraction layer between the business layer and the data access layer, handling common data access logic and improving code reusability.
 - **Cache layer:** Stores frequently accessed data to improve performance and reduce database load.

Distributed Architecture

- This model distributes functionalities across **multiple servers located in different physical locations**.
- This can improve scalability, reliability, and performance by handling heavy traffic and load balancing across servers.
- However, it also requires a more complex infrastructure and expertise to manage.
- **Example:** Large Social Media Platform

Web Services

- Web service is a software system designed to expose specific functionalities over a network, enabling communication and data exchange between applications.
- They follow standardized protocols and use technologies like SOAP or REST to ensure interoperability, meaning they can work seamlessly regardless of the programming languages or platforms involved.
- **Examples:** Weather data service, payment processing service, online map service.

Web Services

- How web services function:

1. **Client Application Sends Request:** An application (the client) makes a request to a web service, specifying what data or functionality it needs.
2. **Web Service Processes Request:** The web service receives the request, interprets it, and performs the necessary actions.
3. **Web Service Sends Response:** The web service sends a response back to the client application, containing the requested data or the results of the performed action.

Web Services

- How web services function: **(Examples)**

- **Payment Processing Service:**

- When you buy something online, the e-commerce website uses a payment processing service to handle the transaction.
 - The website sends your payment information to the payment service, which verifies the payment and sends back a confirmation.

- **Online Map Service:**

- When you use a map app to find directions, the app sends a request to an online map service.
 - The map service responds with the map data, route information, and other location-based services.

Web Services

- Web services play a crucial role in modern web development, enabling various functionalities like:
 - **Data exchange:** Sharing information between different applications, such as product data between an e-commerce platform and a payment gateway.
 - **Remote functionality:** Allowing applications to access functionalities provided by other applications, like weather data retrieval or map services integration.
 - **Modular software development:** Building applications as modular components that can interact and work together through web services.

Thank You